Industrial Robotics

6-DOF Robotic Arm — Design, Simulation, and Implementation

Prepared by:

- Ibrahim Elshami 8819
- Phoebe Emile 9116
- Youssef Fawzy 8680
- Yehia Zahran 8765
- Maya Osama 8582
- Karim Heiba 8929
- Serag Elgewily 8795

1. Mechanical Design (SolidWorks)

1. Kinematic Configuration

The arm follows a classic articulated 6R configuration. The shoulder, elbow, and wrist axes were arranged so that the last three joint axes intersect at a single point, producing a spherical wrist. This choice simplifies the inverse-kinematics decoupling between position and orientation.

Figure 1: Photo of the arm

2. Base — Internal Gear Driven by Stepper

The base is the most heavily loaded joint, since it carries the entire arm and any payload moment. To obtain a high reduction ratio with low backlash and a compact footprint, the base uses an internal (ring) gear fixed to the rotating platform, meshed with a pinion mounted on the stepper-motor shaft. The stepper drives the pinion, the pinion walks around the internal teeth, and the platform rotates relative to the fixed base. Vertical guides were mounted on the base to ensure the stability of the gear-cap assembly in case of larger payload. Guides were equipped with rolling bearings too for smooth movement without friction with the cap surface.

- **Stepper motor model:** Nema 17
- **Pinion / ring teeth:** 10 : 20 teeth
- **Resulting gear ratio:** 2:1 — providing fine angular resolution at the base.

However, after printing and assembling the parts, some challenges were encountered regarding the relative motion between the gears. This has lead to removing the pinion driving gear and fixing the stepper shaft directly to the cap-ring assembly.

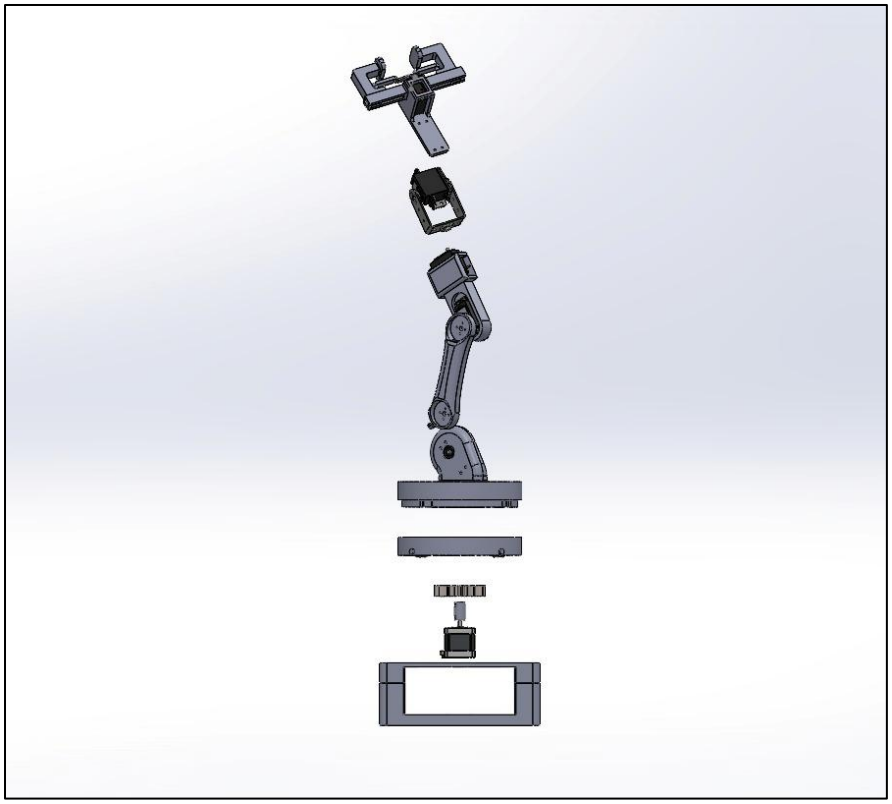


Figure 2: Exploded view

3. Links, Joints, and Servo Actuation

Joints J2 through J6 are actuated by servo motors (mg945) mounted directly at or near the corresponding axis. Each servo is housed in a custom bracket designed in SolidWorks to align its output shaft with the joint axis and to react bending loads onto the mating link rather than the servo body.

All parts, including the upper and lower base and the gears, were 3D printed in black PLA+ filament. Only 2 metal brackets were used to provide the rotation of the end effector around the z-axis. Parts with higher loads, such as the pinion, were printed with higher infill density to ensure they endure the required strength.

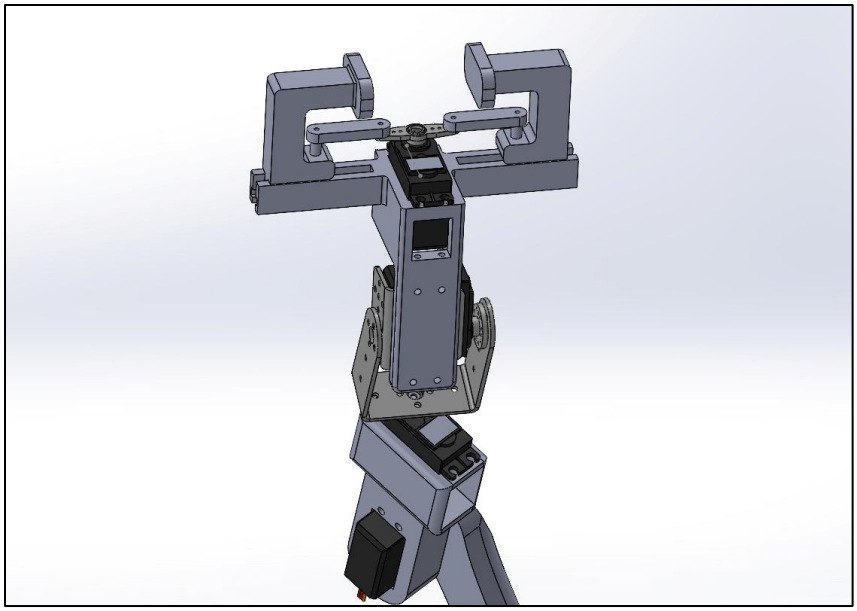


Figure 3: End effector mechanism

4. CAD Output

The completed SolidWorks assembly was used to (i) verify geometric interferences and collision-free motion, (ii) export STL files for 3D printing, and (iii) export the URDF used downstream by ROS 2.

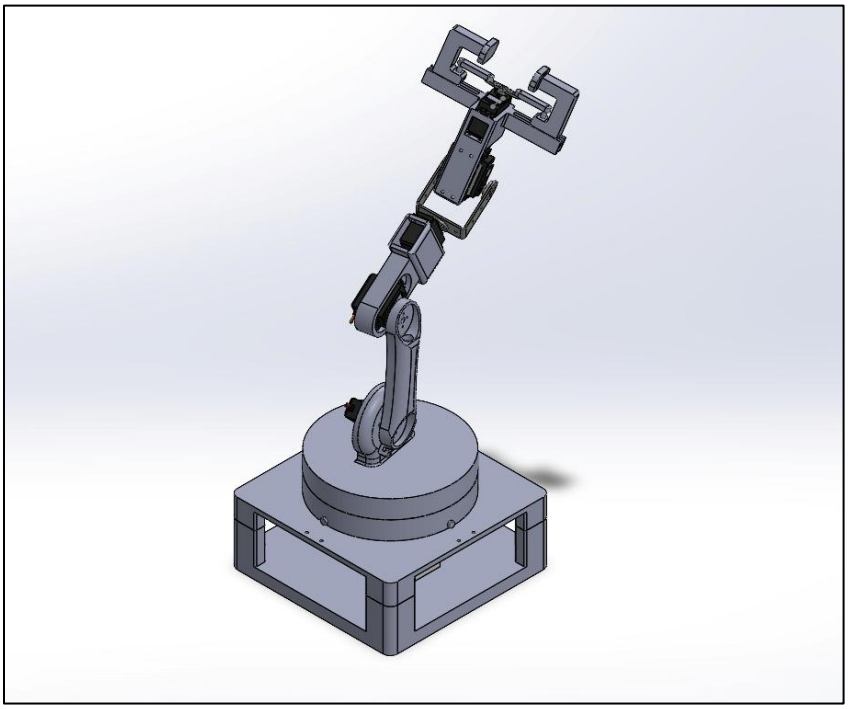


Figure 4: Isometric view of the assembly

2. Simulation in Simscape Multibody

The SolidWorks assembly was first simplified by removing the upper, lower base and the guides to minimize the number of blocks in the Simscape Multibody and only import the effective parts of the assembly to ensure smooth and efficient simulation. The assembly was then imported into MATLAB / Simulink Simscape Multibody using the Simscape Multibody Link exporter. This produced a hierarchical multibody model in which each link is a rigid body and each joint is a Revolute Joint block with its actuation and sensing ports exposed.

1. Model Setup

- The coordinate frames of the robot base and each link of the arm were initially identified in the SolidWorks assembly to match the world frame of the mechanics explorer on Simscape.
- Each revolute joint was configured to be actuated by received motion and outputs a signal that represents its position. Signals are then linked to PS-simulink converter to use them later in the kinematics analysis.
- Inertial properties (mass, center of mass, inertia tensor) were inherited from SolidWorks material assignments and saved in the Assem2_DataFile.m

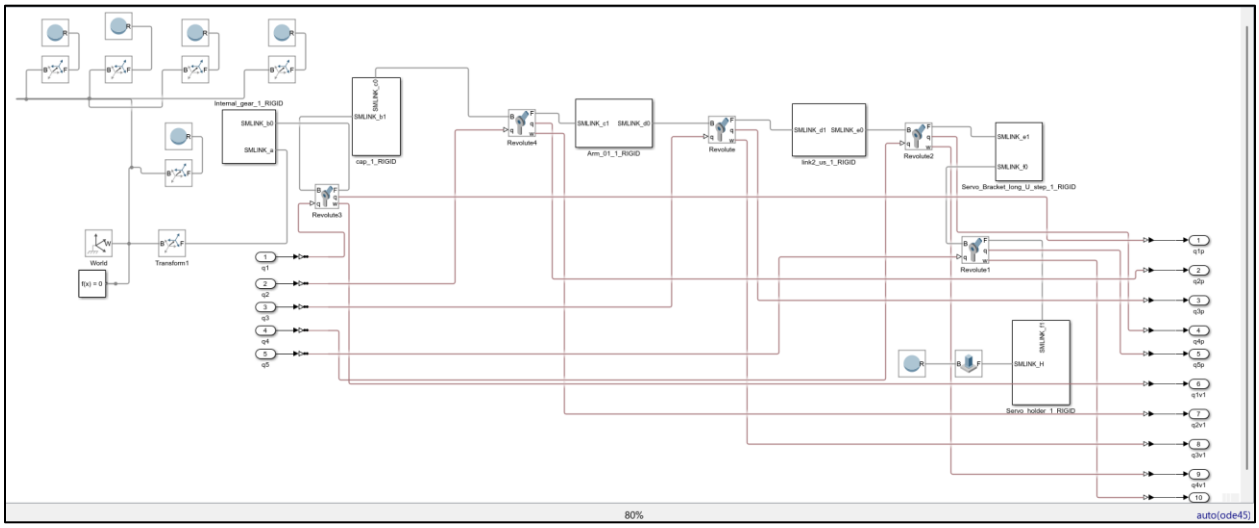


Figure 5: Simscape block diagram

Spheres added relatively to the world frame will be later used for the trajectory planning.

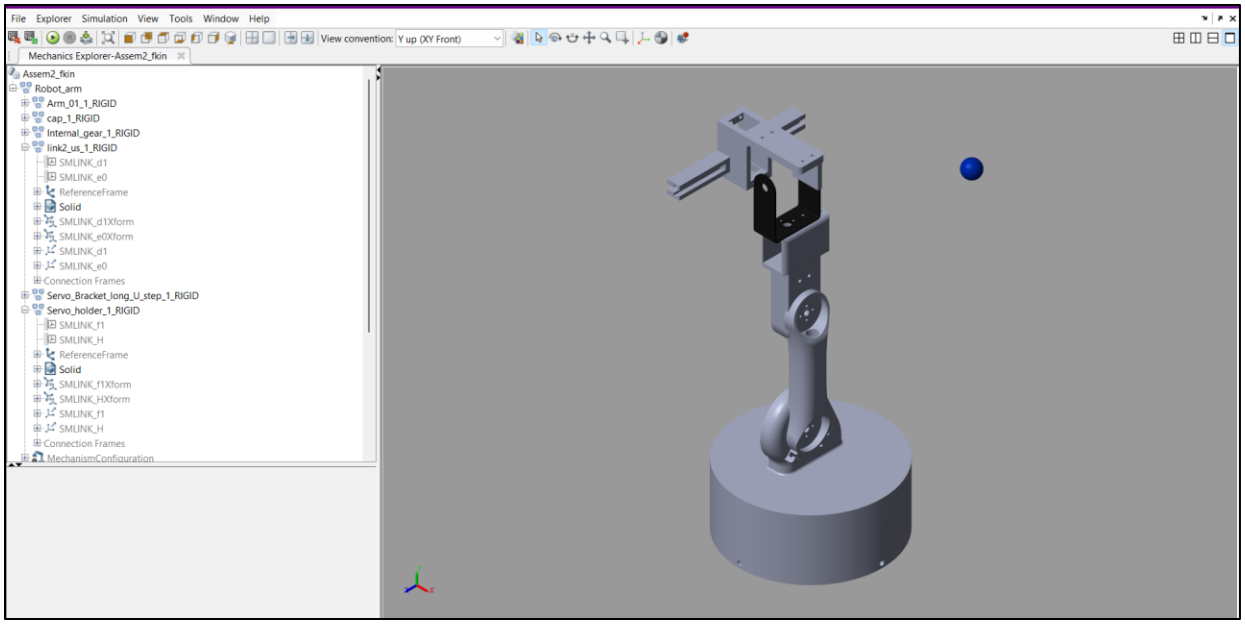


Figure 6: Mechanics explorer view

2. Forward Kinematics

Forward kinematics was implemented by feeding the 5 joint angles into the Revolute Joint blocks through the whole assembly block as shown in the figure and reading the corresponding position of each joint then converting it into a homogeneous transformation with the “Get transform” block. This produced the end-effector position (x, y, z) and orientation (as a rotation matrix and ZYX Euler angles) for any input joint configuration.

The revolute joint controlling the end effector was not implemented in this step just for simplification. The gripper motion could be controlled independently later.

It is noticed that the translation column vector is the same as the resulting end-effector position. The gain blocks were used to convert from radian to degree and vice versa.

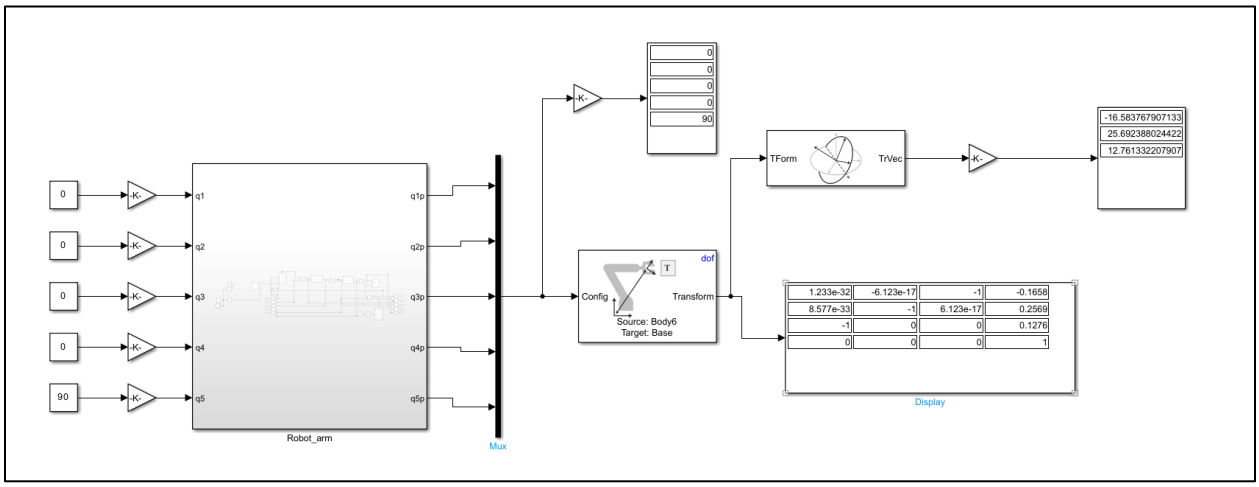


Figure 7: Forward kinematics block diagram

The Denavit–Hartenberg (DH) parameters extracted from the CAD model are summarized below and were used as the reference for all analytical cross-checks.

```
robo =

zack:: 5 axis, RRRRR, stdDH, slowRNE
+---+-----------+-----------+-----------+-----------+-----------+
| j |     theta |         d |         a |     alpha |    offset |
+---+-----------+-----------+-----------+-----------+-----------+
|  1|         q1|         66|          0|     1.5708|          0|
|  2|         q2|          0|        120|          0|          0|
|  3|         q3|          0|         20|    -1.5708|          0|
|  4|         q4|        100|          0|     1.5708|          0|
|  5|         q5|          0|         95|          0|          0|
+---+-----------+-----------+-----------+-----------+-----------+
```

Figure 8: DH table

3. Inverse Kinematics

Inverse kinematics was solved using the “Inverse kinematics” block that receive the required end effector position after converting it into a 4×4 homogeneous matrix and initial guesses for the angles. The angles are then computed and sent to the revolute joints. It is noticed that these angles are matching exactly those read from the outputs of the joints.

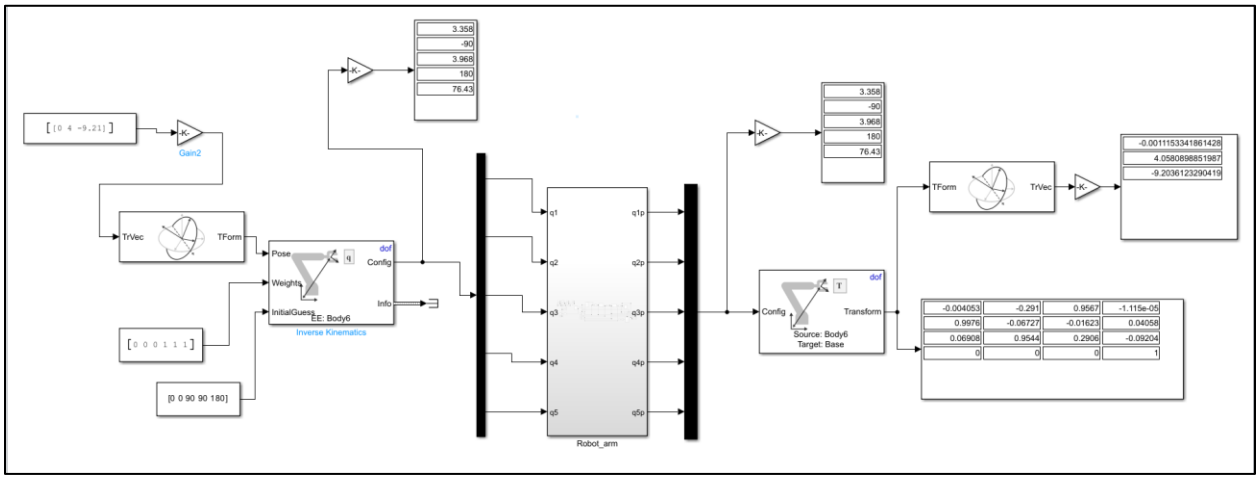


Figure 9: Inverse kinematics block diagram

4. Jacobian Analysis

The Jacobian matrix was simply computed with the “Get Jacobian” block that only needs the angles resulting from the inverse kinematics as input to calculate the 6×6 matrix.

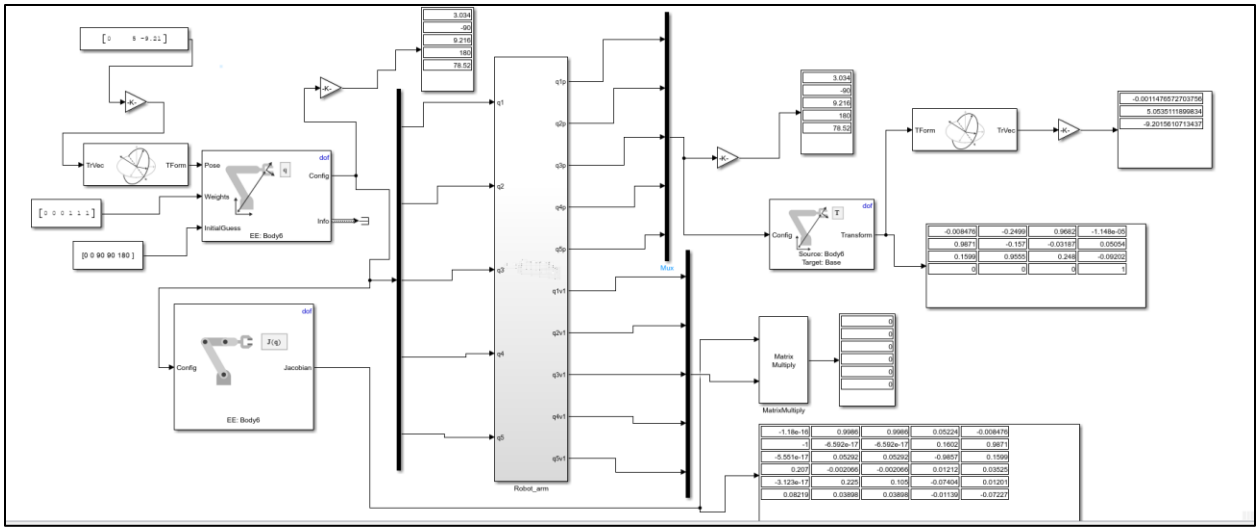


Figure 10: Jacobian analysis block diagram

5. Trajectory Planning

The cubic polynomial trajectory planning method was used to control the arm to move in a path that draws nearly a square. However, this was implemented with 2 different approaches:

- **Using Positions:** a matrix of the required end-effector position was fed to “Polynomial trajectory” block and the output was then sent to the inverse kinematics algorithm to compute the angles.
- **Using angles:** the angles of each revolute joint were fed to the block and the output is sent directly to the joints.

Both generated trajectories were played back through the Simscape model to confirm smooth motion, absence of joint-limit violations, and respect of velocity limits.

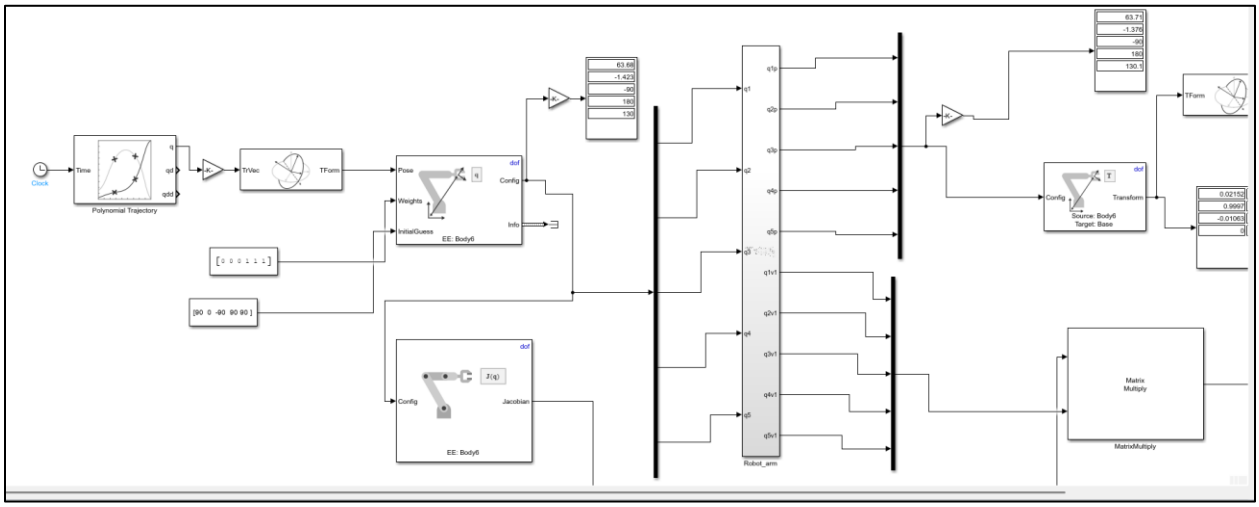


Figure 11: Trajectory planning using positions

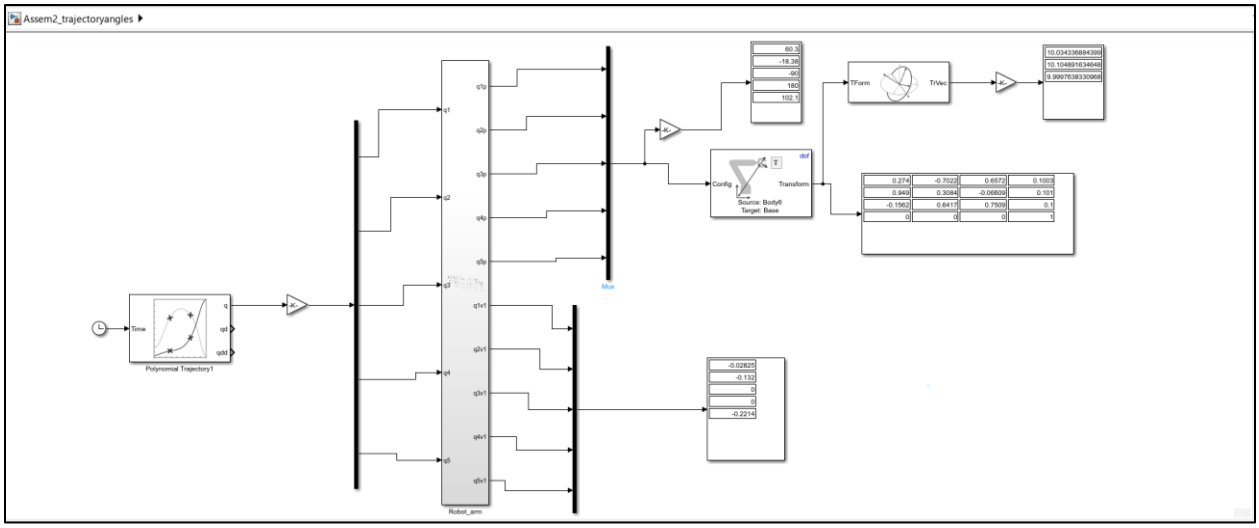


Figure 12: Trajectory planning using angles

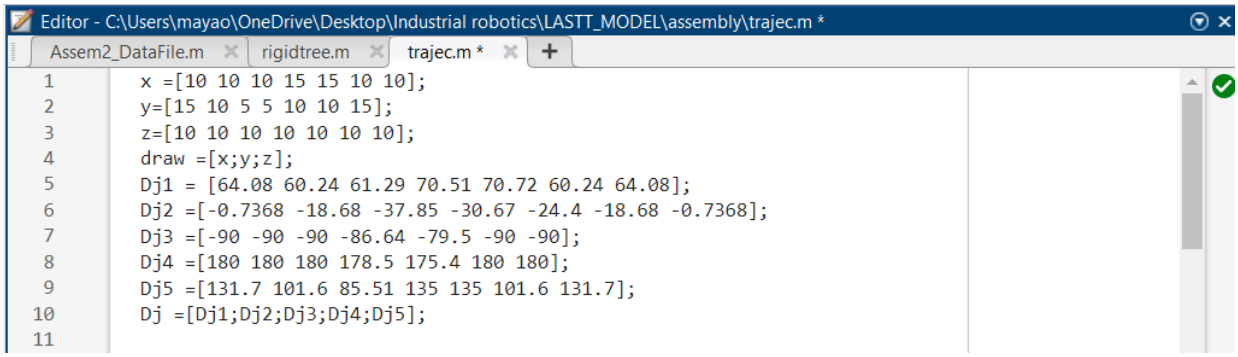


Figure 13: Positions and angles matrices

6. ESP32 deployment on MATLAB:

To publish the generated path to the microcontroller, an add-on hardware package has been installed to establish a connection between the ESP32 and the Simulink model. Blocks called “Servo wiring” are implemented in the model representing each servo with its respective pin on the ESP32. These blocks are fed with a repeating sequence representing the matrices of the angles resulting from the inverse kinematics after adding a constant angle value unique to each one.

The addition of constant angle values is interpreted by compensating the difference between the zero position of each servo in real life and the simulation. Moreover, the limits of the servo angles on MATLAB are ranging from -90 to 90, while the actual limits are from 0 to 180 degrees.

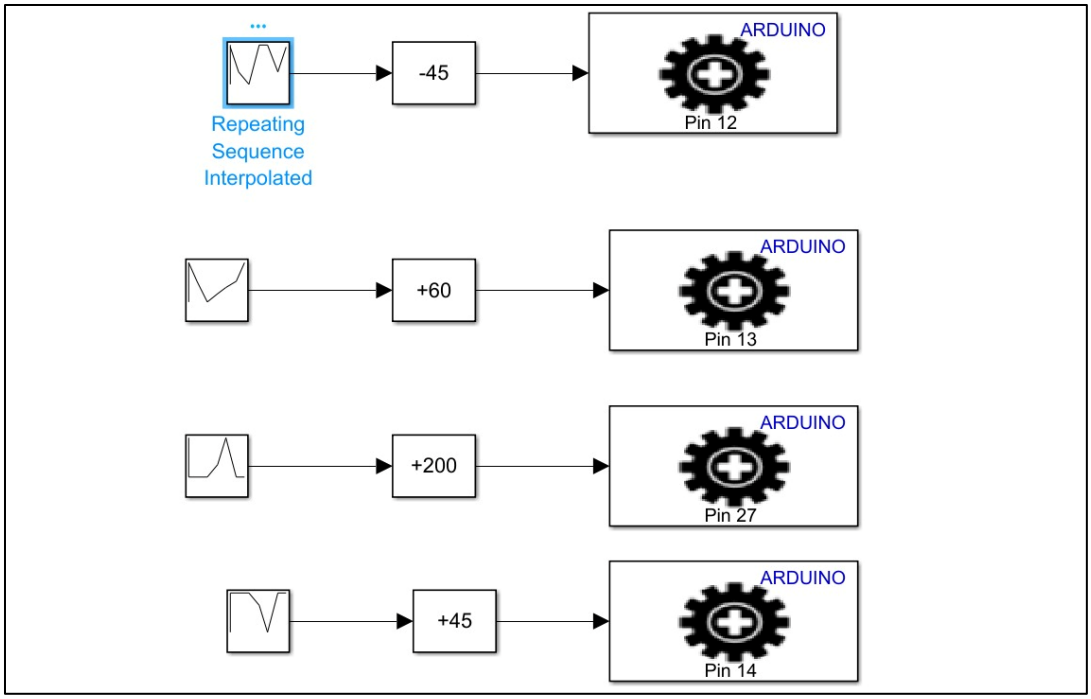


Figure 14: ESP32 control block diagram

3. Kinematics with the Peter Corke Toolbox

To independently validate the Simscape kinematic model, a parallel implementation was written in both Matlab and Python using the Peter Corke Robotics Toolbox (roboticstoolbox-py). The same DH table from Section 2.2 was used to build a SerialLink / DHRobot model of the manipulator.

1. Implementation

- Built a DHRobot object from the modified-DH table of the arm.
- **Forward kinematics:** computed using robot.fkine(q) and compared element-wise with the Simscape Transform Sensor output.
- **Inverse kinematics:** computed using both the closed-form geometric solution and the iterative numerical solver (robot.ikine_LM) provided by the toolbox.
- **Visualization:** the toolbox's built-in 3D plot was used to verify joint configurations and trajectories visually.

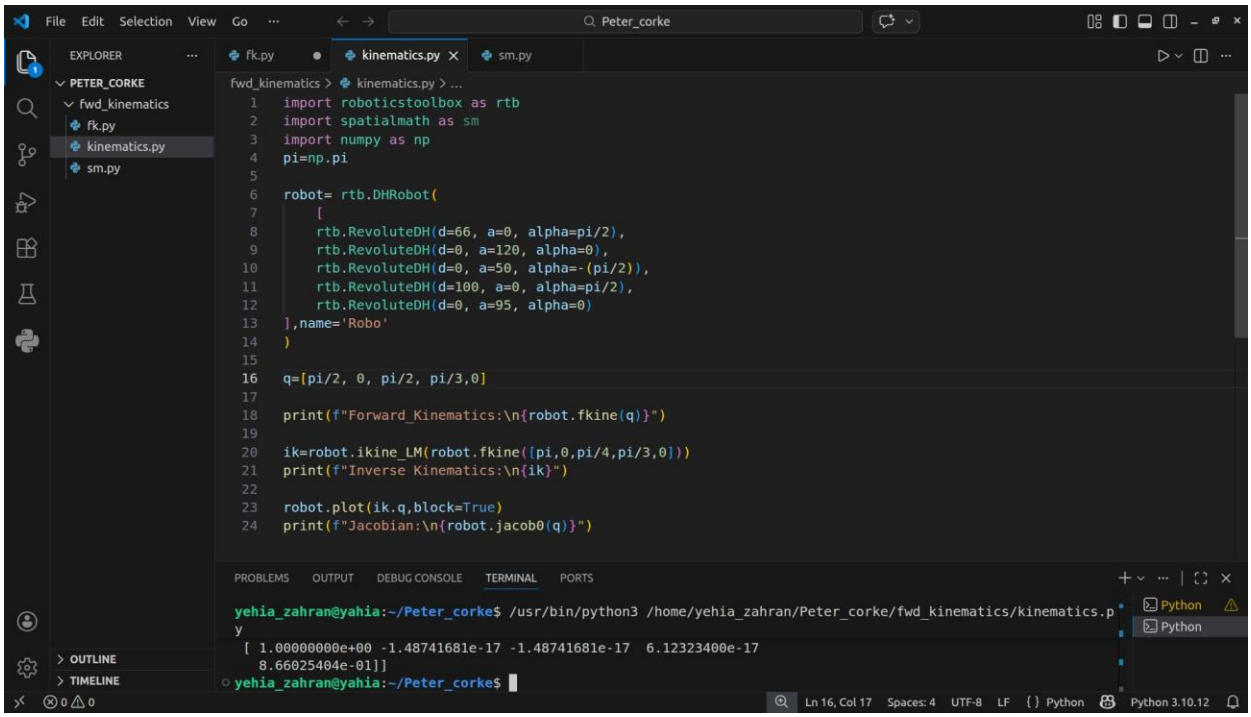


Figure 15: Peter Corke toolbox on Python

```
yehia_zahran@yahia:~/Peter_corke$ /usr/bin/python3 /home/yehia_zahran/Peter_corke/fwd_kinematics/kinematics.py
Forward_Kinematics:
  -0.866     0         0.5      -82.27
   0        -1         0         20
   0.5       0         0.866     163.5
   0         0         0         1


Inverse Kinematics:
IKSolution: q=[-3.142, 0, 0.7854, 1.047, 0], success=True, iterations=22, searches=1, residual=4.27e-10

Jacobian:
[[-2.00000000e+01 -5.18246467e-15  4.59604128e-15 -4.75000000e+01
  -2.90853615e-15]
 [-8.22724134e+01 -9.75000000e+01 -9.75000000e+01 -5.03773238e-15
  -9.50000000e+01]
 [-2.41060648e-15  2.00000000e+01 -1.00000000e+02 -8.22724134e+01
   7.79339912e-16]
 [-1.48741681e-17  1.00000000e+00  1.00000000e+00  6.79815537e-33
   5.00000000e-01]
 [-1.52539319e-32 -6.12323400e-17 -6.12323400e-17 -1.00000000e+00
  -8.20357802e-18]
 [ 1.00000000e+00 -1.48741681e-17 -1.48741681e-17  6.12323400e-17
   8.66025404e-01]]
yehia_zahran@yahia:~/Peter_corke$
```

Figure 16: Kinematics solutions with Peter Corke

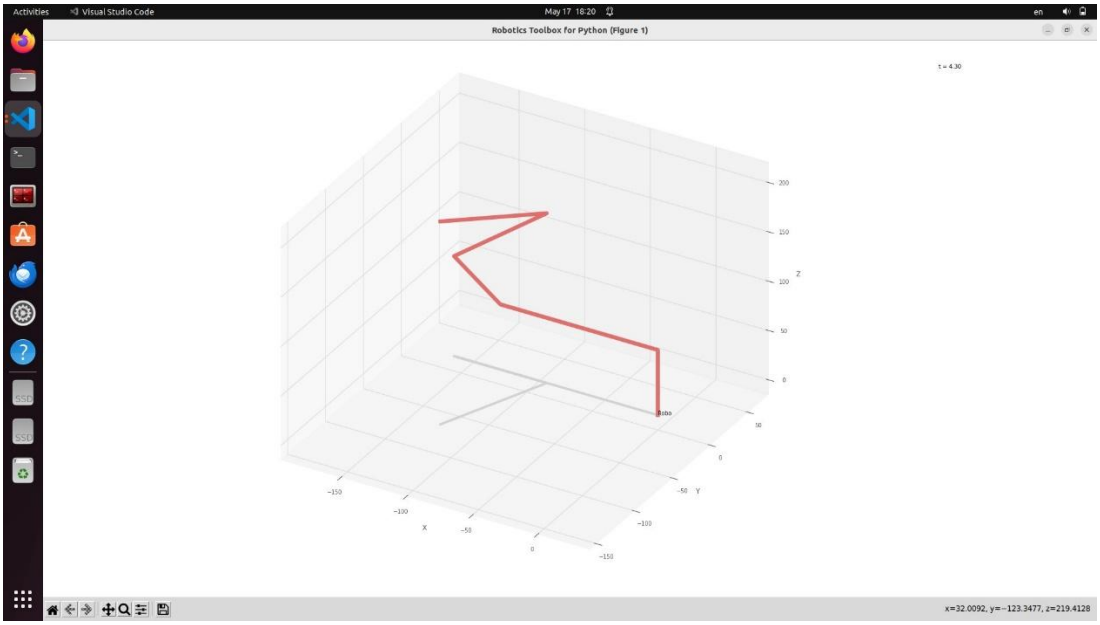


Figure 17: Robot simulation with Peter Corke

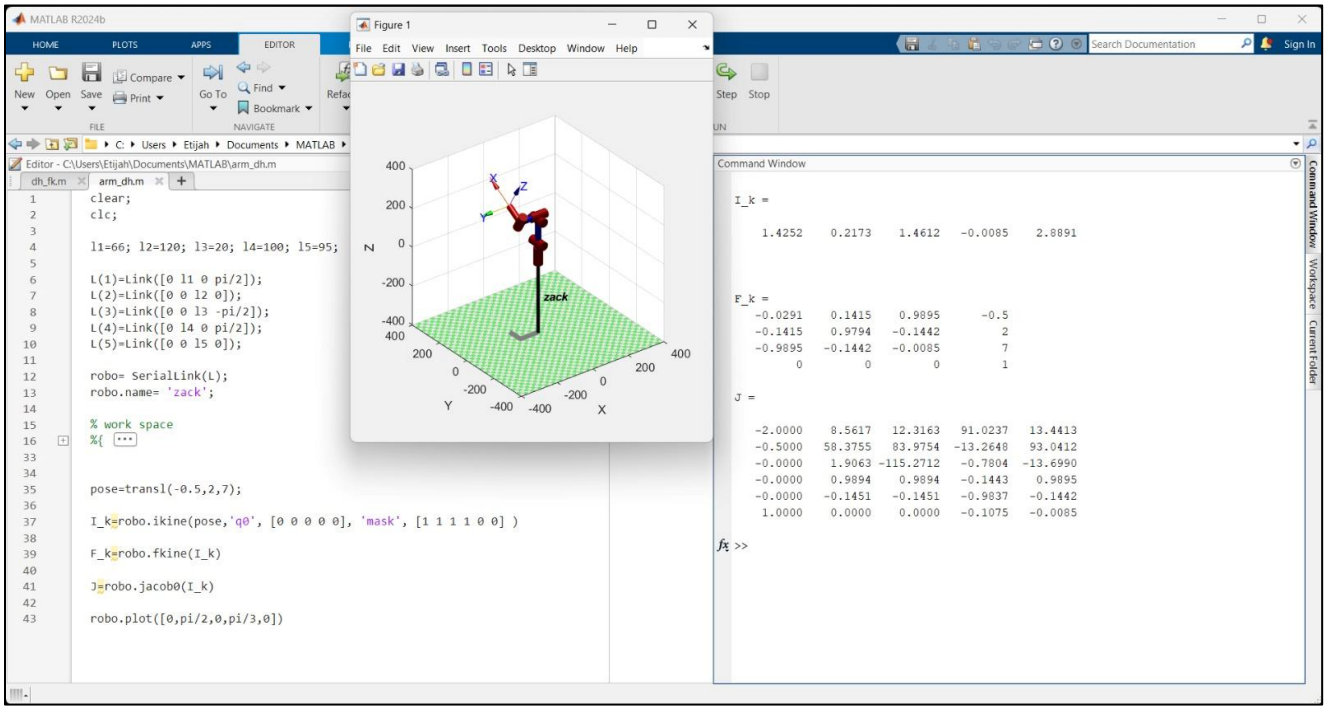


Figure 18: Peter Corke toolbox on Matlab

```
L(1) = Link([0, 66,  0,  pi/2],  'standard');   % Joint 1
L(2) = Link([0, 0,  120, 0],     'standard');   % Joint 2
L(3) = Link([0, 0,  20, -pi/2],  'standard');   % Joint 3
L(4) = Link([0, 100, 0,  pi/2],  'standard');   % Joint 4
L(5) = Link([0, 0,  95,  0],     'standard');   % Joint 5

Rob = SerialLink(L);
Rob.name = 'ASSEM';

disp('Robot successfully created:');
Rob
t = transl(150, 50, 200);                    % Target position
q0 = [0, pi/4, 0, 0, 0];                     % Initial guess
Iinv = Rob.ikine(t, q0, 'mask', [1 1 1 0 0 0]);

if isempty(Iinv)
    disp('No IK solution found. Trying different target...');
    % Try a closer target
    t = transl(100, 80, 150);
    Iinv = Rob.ikine(t, q0, 'mask', [1 1 1 0 0 0]);
end
if ~isempty(Iinv)
    disp('IK Solution Found!');
    disp('Joint angles (degrees):');
    disp(rad2deg(Iinv));

    finv = Rob.fkine(Iinv);
    disp('Achieved Position (mm):');
    disp(finv.t');
else
    disp('Still no solution. Target may be unreachable.');
end
figure;
hold on; grid on;
xlabel('X (mm)'); ylabel('Y (mm)'); zlabel('Z (mm)');
title('5DOF Robot Workspace');
n_points = 50000;

for i = 1:n_points
    q = [ (rand-0.5)*pi*1.8, ...
          (rand)*1.2, ...
          (rand-0.5)*pi, ...
          (rand-0.5)*pi, ...
          (rand-0.5)*pi ];

    T = Rob.fkine(q);
    pos = transl(T);
    plot3(pos(1), pos(2), pos(3), 'r.', 'MarkerSize', 2);
end
axis equal;
view(3);
disp(['Workspace finished with ', num2str(n_points), ' points!']);
```

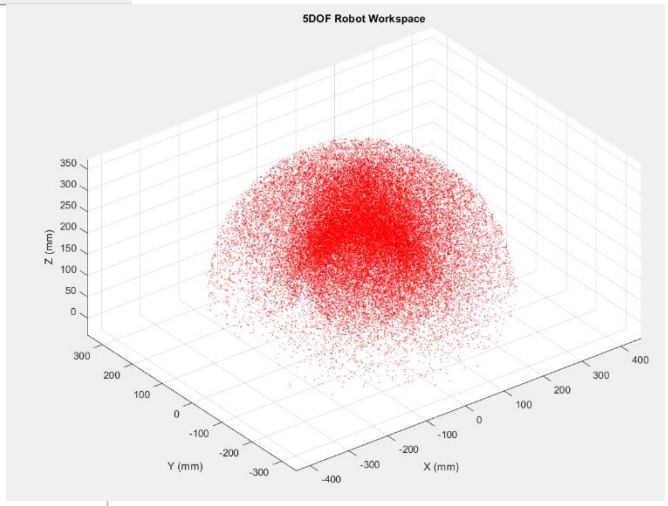


Figure 19: Workspace generation with Peter Corke

4. Electronics and PCB Design

The electrical system of the robot is based on an ESP32 module, an external stepper motor driver, and custom PCBs designed using Altium Designer. The custom electronics were developed to simplify wiring, improve reliability, and organize both the power and control connections inside the system.

The entire system operates from a single 5 V power supply. While the servo motors and ESP32 are powered directly from the 5 V rail, a dedicated boost-converter PCB generates 12 V for the stepper motor driver.

1. ESP32 Carrier PCB

This PCB acts as the main controller carrier and signal distribution board for the system. It hosts the ESP32 module and provides organized routing for both power and control signals. The board distributes the 5 V supply to the servo motors, routes PWM control signals from the ESP32, and connects the stepper driver control pins through dedicated headers.

The use of a custom carrier PCB reduces cable clutter, simplifies assembly, improves signal organization, and makes the overall system easier to debug.

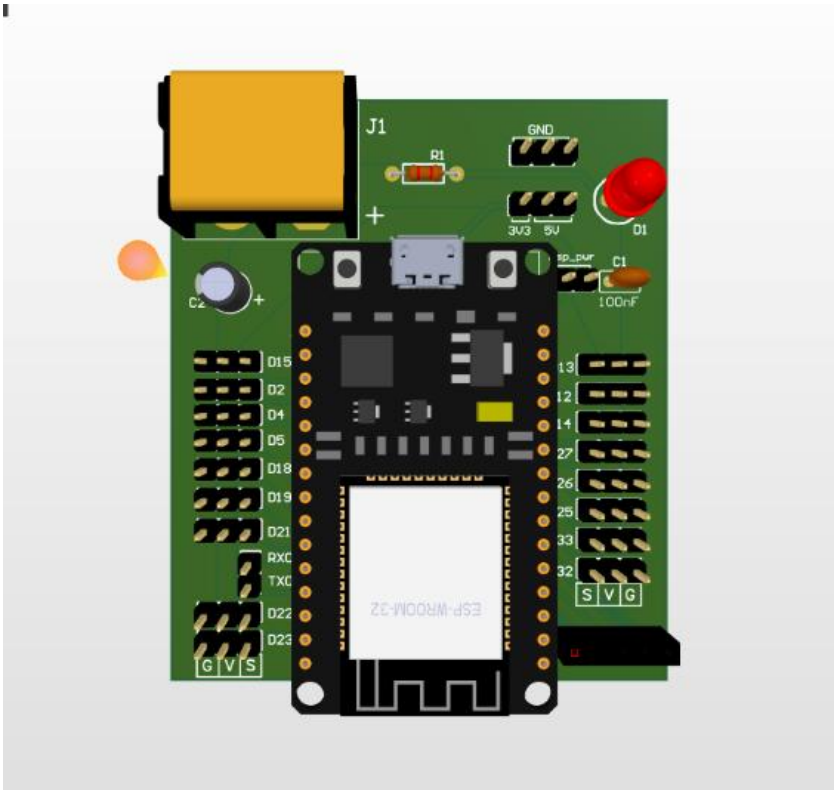


Figure 20: fabricated PCB photo

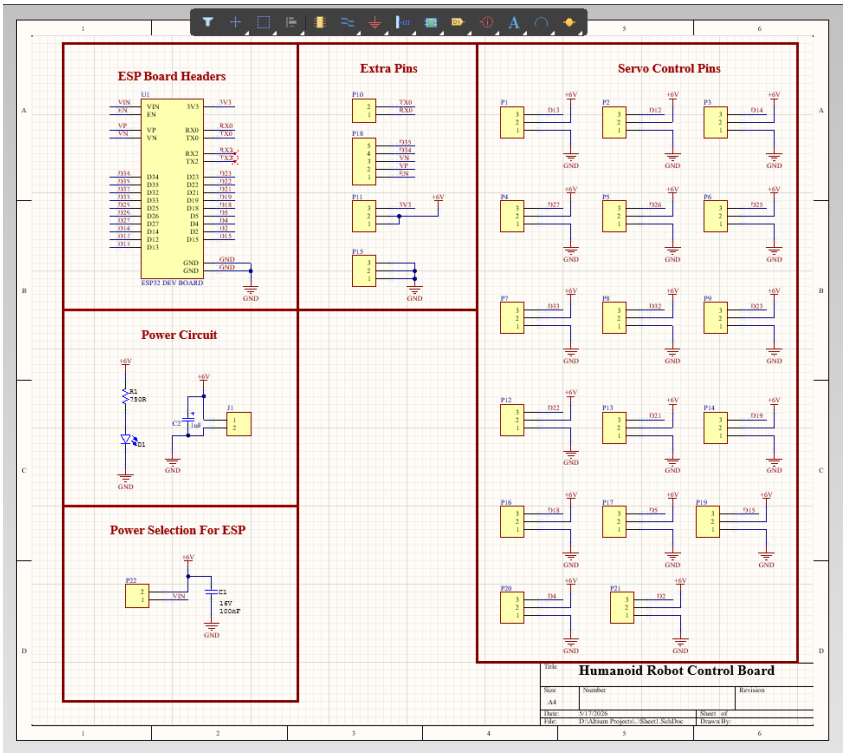


Figure 21: schematic of the ESP32 carrier PCB

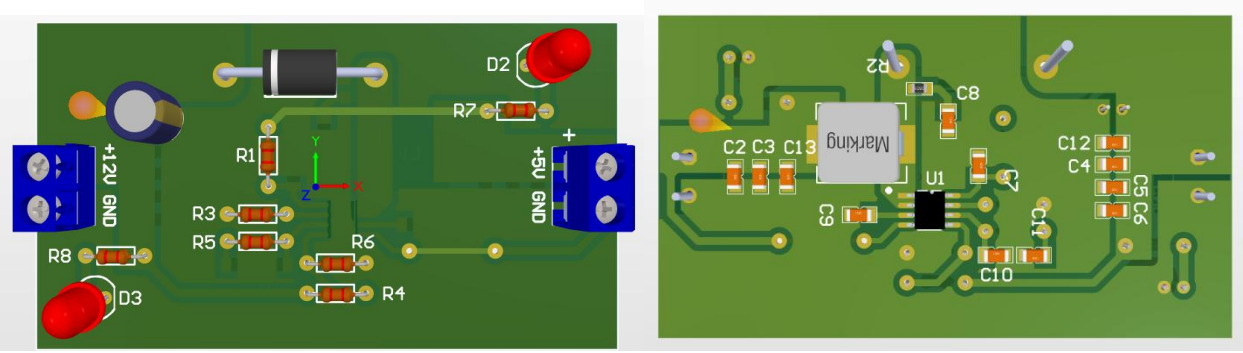
2. Boost Converter PCB

This PCB is a custom DC-DC boost converter designed to generate 12 V from the main 5 V supply using the FP6296XR-G1 IC.

The generated 12 V output is used to power the external stepper motor driver, allowing the entire robotic system to operate from a single power source without requiring an additional supply.

Main features:

- Fixed 400 kHz switching frequency.
- Built-in compensation circuit for improved voltage stability.
- Integrated overcurrent protection.
- Input/output filtering capacitors for ripple reduction.



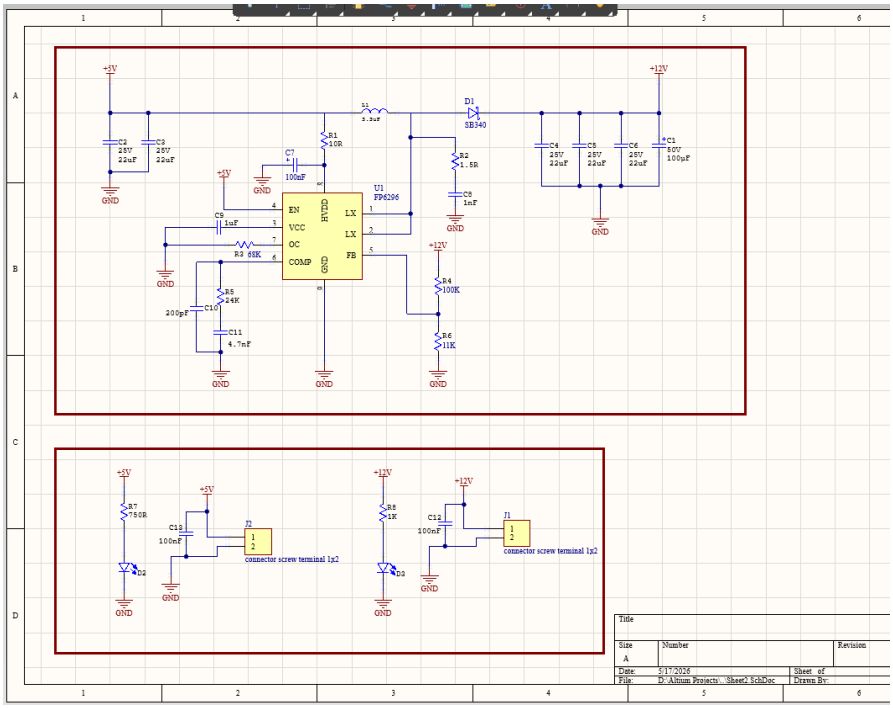


Figure 22: schematic of the boost converter PCB

5. ROS 2 Integration, URDF, Gazebo and MoveIt

1. URDF Development and Validation

The robot arm was designed in SolidWorks and exported to URDF format. Before proceeding to motion planning, the URDF must be validated to ensure the conversion preserved all mechanical properties correctly.

Launch command:ros2 launch robot_bringup display.launch.xml

This launch file performs the following:• Loads the URDF into the robot_state_publisher• Starts RViz with a pre-configured view• Allows visual inspection of joint hierarchies, link transformations, and coordinate frames

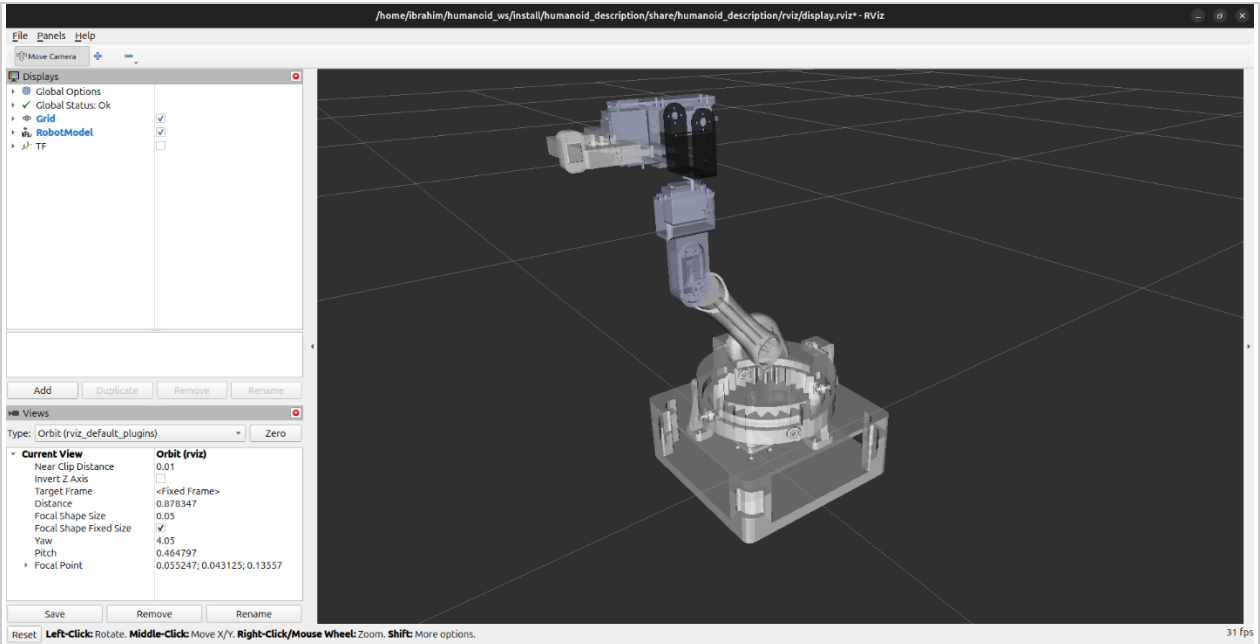


Figure 23: RViz visualization showing robot URDF

2. Motion Planning with MoveIt

Once the URDF is validated, motion planning is performed using MoveIt. The system uses the OMPL (Open Motion Planning Library) planner with KDL (Kinematics and Dynamics Library) for inverse kinematics solving.

The combined launch file brings up the following components:• robot_state_publisher: Publishes the robot's kinematic tree to /tf• ros2_control_node: Manages hardware interfaces and controllers• Controller spawners: Start the joint state broadcaster and arm trajectory controller• move_group: MoveIt's core planning and execution node• RViz: Interactive interface for planning and visualization

3. Simulation in Gazebo

Before deploying to real hardware, all planned motions are tested in Gazebo, a physics-based robot simulator. Gazebo loads the same URDF with additional physics properties

and validates that the planned trajectories are executable under realistic conditions.

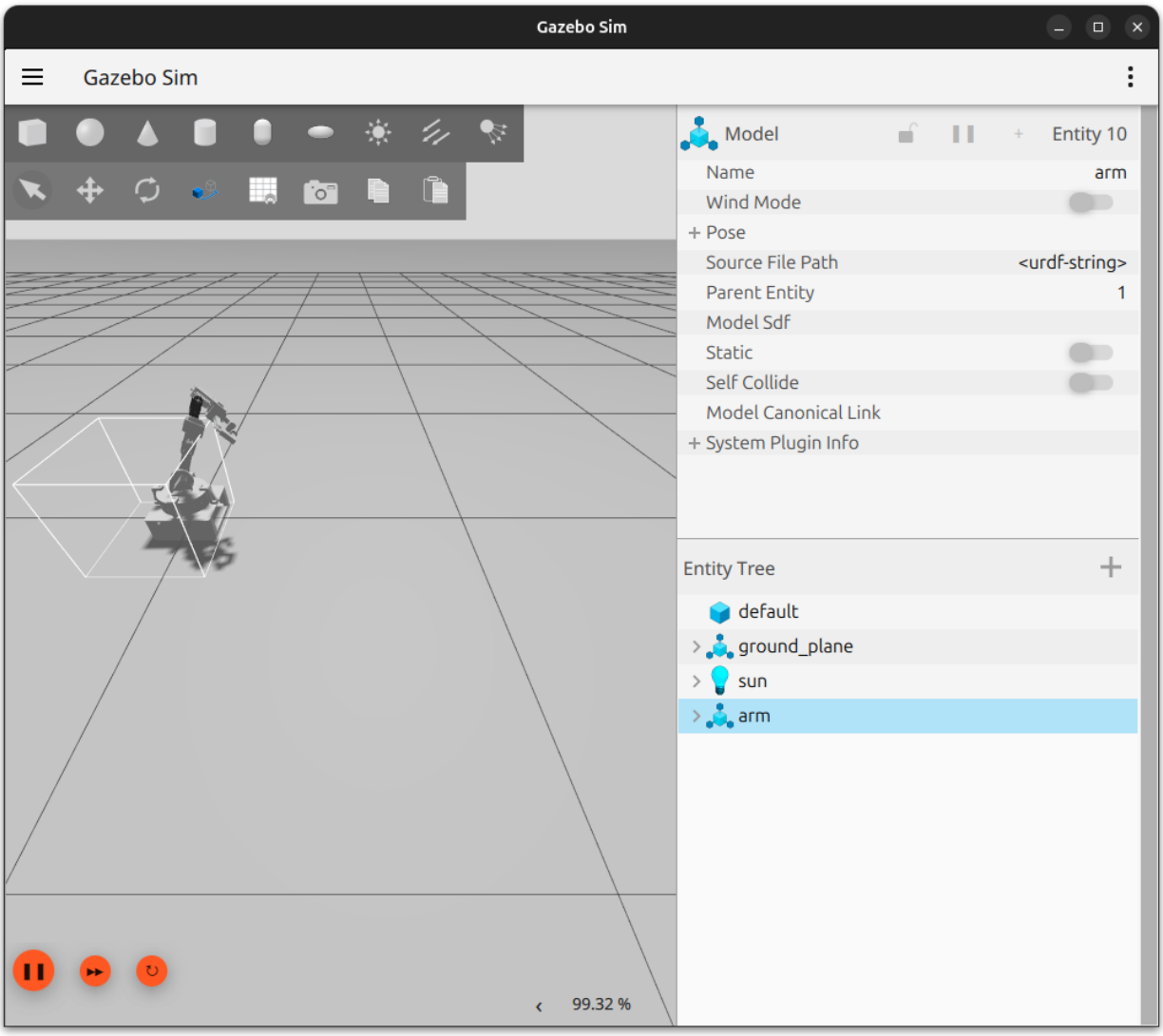


Figure 24: Gazebo simulation executing a planned trajectory

Benefits of Gazebo simulation:• Safe testing without risk to physical hardware• Visualization of dynamics (gravity, inertia, friction)• Debugging of controller gains and timing• Validation of workspace reachability and collision avoidance

4. Gripper Control with PS5 Controller

The robot gripper is controlled using a PS5 controller through ROS2's joy node. This provides intuitive manual control of the end effector while the arm follows planned trajectories.

The joy node publishes button and axis data to the /joy topic, which is then processed by a custom node that translates controller inputs into gripper commands.

Implementation:• joy node interfaces with the PS5 controller via Bluetooth• Custom gripper controller node subscribes to /joy topic• Button presses are mapped to gripper open/close commands• Commands are sent to the ESP32 via the serial bridge

5. Hardware Deployment via ESP32

After validating motions in simulation, the same trajectories are sent to the physical robot arm. A custom ROS2 node subscribes to the /joint_states topic and forwards joint angles to an ESP32 microcontroller via serial communication.

Serial Communication Node

A dedicated ROS2 node acts as a bridge between the high-level ROS2 system and the low-level hardware:

Functionality:• Subscribes to /joint_states topic• Extracts joint position data• Formats commands for ESP32 (comma-separated angles)• Transmits via UART at 115200 baud• Receives acknowledgment/feedback from ESP32

ESP32 Responsibilities

The ESP32 microcontroller handles the low-level actuation:• Receives joint angle commands via UART• Converts angles to servo PWM signals• Drives servo motors with appropriate timing• Implements safety limits to prevent mechanical damage• Optionally sends position feedback to ROS2

Communication Protocol

• Baud rate: 115200• Data format: Comma-separated angles• Example: "45.2,90.0,-30.5,120.0\n"

6. Results and Achievements

✓ Complete URDF model extracted from SolidWorks✓ Motion planning with collision avoidance in MoveIt✓ Gazebo simulation matching physical robot kinematics✓ Real-time joint state publishing and visualization✓ PS5 controller integration for gripper control✓ Serial communication bridge to ESP32 hardware✓ Unified launch files for integrated workflows

6. Conclusion

This project successfully demonstrated a complete robotics software stack, from CAD modeling to hardware deployment. By integrating MoveIt for planning, Gazebo for simulation, and custom nodes for hardware interfacing, we created a robust system that bridges the gap between virtual and physical robotics.

The workflow validates the importance of simulation in robotics development—enabling safe, rapid iteration before deploying to expensive hardware. The ROS2 ecosystem provided the modular architecture necessary to compartmentalize functionality while maintaining seamless integration between different subsystems.

The addition of PS5 controller integration for gripper control demonstrated how modern gaming peripherals can be effectively used as intuitive human-robot interfaces. The serial communication bridge to ESP32 showcased how ROS2 can be extended to work with low-cost embedded systems, making advanced robotics capabilities accessible without expensive hardware.

7. Appendix: Running the Project

1 Launch Simulation + MoveIt

cd ~/robot_arm && source install/setup.bash && ros2 launch arm_description moevit_gazebo.launch.py

2 Launch URDF Visualization

ros2 launch arm_description display.launch.xml

3 Start Serial Communication Node

ros2 run arm_control control_serial

4 Start Joy Node for PS5 Controller

ros2 run joy joy_node